Resumen de Sistemas Operativos

🎨 *Leyenda de colores*

- Amarillo → Definiciones formales
- Verde → Ventajas / beneficios
- Rojo → Desventajas / riesgos / problemas
- Celeste → Términos técnicos clave (primera aparición del concepto)
- Naranja → Datos numéricos, ejemplos y ejercicios resueltos
- Violeta → Comparaciones y conclusiones

---

1. Película: Revolution OS

1.1. ¿Qué es un sistema operativo? ¿Qué hace?

> ***Definición:*** *Capa de software que el usuario, idealmente, nunca debería ver directamente.*

- **Misión principal:** Ayudar a que otros programas funcionen.
- **Funcionamiento:** No realiza tareas por sí solo; espera a que los programas le pidan recursos (como acceder a un archivo o conectarse al exterior).
- **Propósito:** Facilitar el trabajo de los programadores gestionando el hardware.

1.2. ¿Qué elementos componen un Sistema Operativo?

Según el proyecto GNU, se necesitan cientos de programas para completar el sistema, destacando:

- **Kernel (Núcleo):** El programa que asigna recursos a todos los demás.
- **Compilador:** Traduce el código fuente (legible para humanos) a números que la máquina entiende.
- **Debugger (Depurador):** Para encontrar y corregir errores.
- **Editores y formateadores de texto.**
- **Sistemas de correo (Mailers).**

1.3. ¿Qué diferencias tenían los kernels Linux y HURD?

La diferencia radica principalmente en su arquitectura y facilidad de desarrollo:

- **Linux (Kernel Monolítico):** El sistema operativo es una única entidad indivisible. Linus Torvalds lo desarrolló rápidamente usando métodos de diseño tradicionales.
- **HURD (Microkernel):** El núcleo es una colección de servidores que se comunican entre sí mediante protocolos comunes. Aunque es más avanzado y potente, resultó extremadamente difícil de depurar debido a que los mensajes entre programas ocurren de forma asincrónica, retrasando su funcionamiento por años.

1.4. ¿Qué es Software Privativo o Propietario?

- **Definición:** Modelo donde las empresas "cierran" su software, prohibiendo a los usuarios ver o modificar el código fuente, incluso si lo necesitan para sus aplicaciones.
- **Filosofía:** Se basa en controlar y dominar al usuario, exigiéndole firmar contratos y promesas de no compartir el programa con nadie más (Microsoft fue pionera de este esquema).

1.5. ¿Qué es Open Source (Código Abierto)?

- **Concepto:** Una forma de colaborar en software sin las trabas de la propiedad intelectual o contratos legales complejos.
- **Claves:** Se define por el acceso al código fuente y por otorgar nueve derechos específicos, siendo el primero la **libre redistribución** (compartirlo con o sin costo).

1.6. ¿Qué diferencias hay entre Free Software y Open Source Software?

Aunque comparten bases técnicas, la diferencia es filosófica y de marketing:

- **Free Software (Software Libre):** Liderado por Richard Stallman. Se enfoca en la ética y la libertad social; considera que la libertad de cooperar es esencial para una buena sociedad y que todo el software debería ser libre.
- **Open Source:** Liderado por Eric Raymond y Linus Torvalds. Se enfoca en las ventajas prácticas (potencia, fiabilidad) y es un término creado para ser más atractivo y amigable para las empresas al evitar la palabra "free" (gratis/libre).
- **Dato clave:** Bruce Perens menciona en la película: *"Richard quiere que todo el software sea libre y yo creo que el software libre y el no-libre deberían coexistir"*.

1.7. ¿Qué es el CopyLeft?

> ***Definición:*** *Técnica legal que utiliza el derecho de autor (copyright) "al revés".*

- **Funcionamiento:** En lugar de usar el copyright para restringir, el autor da permiso para redistribuir, cambiar y añadir al software.
- **Condición innegociable:** Cualquier versión redistribuida debe mantener estos mismos términos y libertades, asegurando que la libertad sea un derecho inalienable que acompaña al software.

1.8. ¿Qué quiso expresar Eric Raymond con "La catedral y el Bazar"?

Raymond contrastó dos estilos opuestos de desarrollo de software:

- **La Catedral:** El modelo convencional cerrado. Grupos pequeños, jerárquicos, con especificaciones rígidas y lanzamientos muy espaciados en el tiempo.
- **El Bazar:** El modelo de Linux. Descentralizado, con lanzamientos frecuentes y una revisión masiva y constante de pares (peer review) por parte de desarrolladores ajenos al proyecto original.

1.9. ¿Qué riesgos se corren al permitir que una empresa "pervierta" los estándares abiertos?

El riesgo principal es el **monopolio** y el **bloqueo (lock-in)**:

- Si una empresa domina el mercado y altera estándares abiertos (como HTTP o HTML) para hacerlos propietarios, los convierte en dispositivos de bloqueo.
- Esto le permite expulsar a la competencia y obligar a todos los usuarios y servidores a depender exclusivamente de su tecnología.

2. Procesos (Parte 1)

2.1. Defina proceso

> ***Definición:*** *Programa en ejecución y, de forma más precisa, la unidad de procesamiento gestionada por el sistema operativo.*

- **Requisito:** Para constituir un proceso, un programa necesita haber sido iniciado por el SO.
- **Nota:** Un proceso no necesariamente está en ejecución todo el tiempo durante su existencia.

2.2. Estados principales de un proceso

El ciclo de vida de un proceso consta de varios estados principales: **Nuevo**, **Listo**, **Ejecución**, **Espera (Bloqueado)** y **Salir**.

Explicación de cada estado

- **Nuevo (New):** El proceso se acaba de crear pero aún no tiene recursos asignados. El SO prepara el `PCB` y reserva memoria. Pasa a *Listo* cuando se le asignan los recursos y se copia el programa desde el disco a la `RAM`.
- **Listo (Ready):** El proceso está en una cola de listos, esperando que el planificador le asigne tiempo de `CPU`. Tiene todos los recursos necesarios y está preparado para ejecutarse.
- **Ejecución (Run):** El proceso tiene el control de la CPU y ejecuta sus instrucciones. Puede salir por finalización, ser interrumpido (vuelve a *Listo*), o bloquearse en espera de algún recurso.
- **Espera/Bloqueado (Wait/Blocked):** El proceso no puede continuar porque requiere un evento externo (entrada/salida, lectura de disco, teclado). Libera la CPU y vuelve a *Listo* cuando obtiene lo solicitado.
- **Salir (Exit):** El proceso finalizó su ejecución o fue abortado por un error. El SO libera su memoria y destruye sus estructuras.

2.3. Uso de la pila del sistema en interrupciones o llamadas al sistema

Al transferir el control al SO, se utiliza una **`pila del sistema`** (pila de núcleo) distinta a la pila de usuario del proceso.

Razones fundamentales

- **Seguridad y Aislamiento:** El SO opera en modo núcleo (con control total del hardware) mientras que el proceso opera en modo usuario (restringido). Usar pilas separadas impide manipulaciones maliciosas de la pila del kernel.
- **Fiabilidad del Contexto:** Permite guardar de manera segura los registros y el estado del proceso interrumpido.
- **Prevención de Fallos:** Protege al sistema operativo de desbordamientos de pila (stack overflow) u otros errores graves ocurridos en el programa de usuario.

2.4. Actividades principales del SO en la gestión de procesos

- **Creación de procesos:** Elaboración del PCB (Bloque de Control de Proceso), asignación de memoria física e inicialización del espacio de direcciones.
- **Planificación de la CPU:** Selección del proceso de la cola de listos que tomará el procesador según los algoritmos de planificación.
- **Gestión de ejecución y estados:** Control de las transiciones (ej. mover a bloqueado si espera E/S, o a listo tras una interrupción).
- **Terminación de procesos:** Liberación de memoria y recursos asignados, y eliminación del PCB del sistema.

2.5. Características y causas de un proceso en estado suspendido

Características principales

- **Ubicación física:** La imagen de memoria del proceso es trasladada a la memoria secundaria (swap en disco) para liberar RAM.
- **Suspensión de ejecución:** El proceso no puede ser planificado para usar la CPU mientras esté en este estado.
- **Preservación:** El SO mantiene el PCB del proceso en la memoria principal para no perder su control.
- **Variantes:** Se divide en *Suspendido-Bloqueado* (espera un evento en disco) y *Suspendido-Listo* (listo para ejecutar pero reside en disco).

Causas de suspensión

- **Falta de memoria principal:** El SO necesita RAM para otros procesos activos o para admitir un proceso nuevo.
- **Decisión del planificador:** El SO mueve a disco procesos bloqueados de larga duración para mejorar el rendimiento.
- **Petición del usuario o sistema:** Pausa manual del programa o suspensión automática ante sobrecargas.

2.6. Intercambio de procesos (Swapping)

> ***Definición:*** *Técnica de gestión de memoria (denominada swapping) que consiste en mover la imagen de un proceso desde la RAM hacia el almacenamiento secundario y viceversa.*

Flujo de ejecución

1. **Saturación de RAM:** Los procesos residentes se encuentran bloqueados en espera de eventos.
2. **Expulsión (Swap Out):** El SO suspende procesos enviándolos al disco.
3. **Admisión (Swap In):** El SO trae desde el disco a un proceso listo (estado Listo/Suspendido) para aprovechar la CPU.

2.7. Esquema general de tablas del SO y estructura del BCP

El SO organiza su control a través de tablas diseñadas para eficiencia y seguridad:

- **Eficiencia (Estructura Estática):** La tabla de procesos se compone de BCPs de tamaño idéntico para acelerar el direccionamiento.
- **Información Variable:** Datos dinámicos (como tablas de páginas) no residen en el BCP, sino que se acceden mediante punteros de control (como el RIED).
- **Información Compartida:** Datos que comparten varios procesos (como la posición de un archivo abierto) se mantienen en tablas externas globales.

Estructura de los Procesos (Detalle del BCP)

A. Contenido Interno (Información Estática/Control)

- **Identificadores:** PID del proceso, PPID (proceso padre) e ID de usuario.
- **RIED (Registro Identificador del Espacio de Direccionamiento):** Para gestionar la memoria.
- **Descripción de Segmentos:** Dirección de inicio, tamaño, zona de crecimiento y puntero a subtabla de páginas.
- **Tabla de descriptores de archivos (fd):** Lista interna de descriptores abiertos.

B. Contenido Externo (Información Variable o Compartida)

- **Subtabla de páginas:** No reside en el BCP por su tamaño variable; el BCP tiene un puntero a ella.

2.8. Ejercicio: Traza combinada de Round-Robin

Suponga que una computadora tiene tres procesos (P_1, P_2, P_3) para ejecutar. El planificador utiliza un algoritmo **Round-Robin** en el orden P_1, P_2, P_3. El quantum es de 16 ciclos de reloj. La CPU ejecuta 1 instrucción cada 4 ciclos de reloj (máximo 4 instrucciones por turno). P_2 tiene una E/S en la dirección 723A que tarda 10 ciclos. Al ocurrir una E/S, el planificador pasa a otro proceso y retorna cuando finaliza la E/S.

Traza combinada resultante: 1. `0100 - 0101 - 0103 - 0105` (P_1) - Vuelve a la cola de listos (quantum vencido). 2. `7231 - 7234 - 7236 - 7238 - 723A` (P_2) - Bloqueo por E/S en `723A` (P2 sale de CPU). 3. `8021 - 8022 - 8023 - 8024` (P_3) - Vuelve a la cola de listos (quantum vencido). 4. `0109 - 010B - 010D` (P_1) - P1 finaliza su ejecución. 5. `8025` (P_3) - P3 finaliza su ejecución. 6. `723C - 723D - 723F` (P_2) - P2 finaliza su ejecución (E/S terminada).

2.9. Eventos que producen la creación de procesos

Existen cuatro eventos principales que provocan la creación de un proceso:

1. **Arranque del sistema:** Inicialización de procesos del sistema y servicios en segundo plano.
2. **Sesión interactiva (Petición de usuario):** Inicio de sesión de un usuario o ejecución manual de un programa (haciendo clic o comando).
3. **Servicio del sistema operativo:** Creación transparente de un proceso para realizar una tarea en nombre de un programa de usuario.
4. **Creado por un proceso existente (Hijo):** Llamada al sistema desde un proceso activo para explotar el paralelismo y modularidad.

2.10. ¿Cómo se llama la creación de un proceso por otro?

- **Jerarquía de Procesos:** Estructura organizativa de relaciones de creación de procesos.
- **Proceso Padre:** El proceso creador (ejecuta la llamada fork).
- **Proceso Hijo:** El proceso clonado o creado.

2.11. Ejemplos de terminación de procesos

Un proceso finaliza por tres causas principales:

- **Finalización normal:** El programa ha terminado con éxito todas sus instrucciones.
- **Condiciones de error (Falla fatal):** División por cero, violaciones de direccionamiento de memoria, o instrucciones inválidas.
- **Intervención externa:** El proceso padre o el usuario deciden terminarlo de forma forzosa.

2.12. Diferenciación de los estados Listo y Bloqueado

El estado "No Ejecutándose" se divide en dos:

- **Listo:** El proceso está cargado en memoria y preparado para usar la CPU en cuanto el planificador le asigne su turno.
- **Bloqueado:** El proceso está esperando que finalice una operación de E/S o evento externo, por lo que no puede ejecutarse aunque la CPU quede libre.

2.13. Ejercicio: Estados de la Traza Combinada

Tiempo (Instrucciones)	Proceso Ejecutando	Estado P1	Estado P2	Estado P3
$1 - 4$	P1	Ejecución	Listo	Listo
$5 - 9$	P2	Listo	Ejecución (Bloquea en 9)	Listo
$10 - 13$	P3	Listo	Bloqueado (E/S)	Ejecución
$14 - 16$	P1	Ejecución (Termina)	Bloqueado (E/S)	Listo
17	P3	Terminado	Listo (E/S Termina)	Ejecución (Termina)
$18 - 20$	P2	Terminado	Ejecución (Termina)	Terminado

2.14. Razón de incorporar el estado Suspendido

La razón principal de incorporar el estado suspendido es optimizar el uso de la memoria principal mediante el intercambio (swapping):

- **Problema:** Si todos los procesos en RAM se bloquean esperando eventos de E/S, la CPU queda ociosa.
- **Solución:** El SO suspende procesos moviendo su contenido de RAM a disco magnético.
- **Beneficio:** Libera RAM para admitir procesos listos o nuevos, incrementando la utilización global del procesador.

2.15. Modelo de 7 Estados (con estados suspendidos)

El modelo de 7 estados incorpora **Bloqueado-Suspendido** (esperando un suceso en disco) y **Listo-Suspendido** (listo para ejecutar pero en disco), permitiendo suspender y activar procesos según el espacio en memoria RAM.

2.16. Estructuras de control más importantes del SO

El SO de la computadora gestiona los recursos a través de cuatro tablas de control principales:

- **Tabla de procesos:** Almacena los BCPs de los procesos activos, rastreando su estado y recursos.
- **Tablas de memoria:** Registran el espacio libre y ocupado en RAM (tablas de páginas/segmentos).
- **Tablas de E/S:** Monitorean el estado de los periféricos y peticiones de E/S pendientes.
- **Tablas de archivos:** Controlan descriptores de archivos abiertos y accesos a disco.

2.17. Bloque de Control de Proceso (BCP)

El Bloque de Control de Proceso (BCP) contiene:

1. **Identificadores:** PID del proceso, PPID (padre) e identificadores de usuario.
2. **Estado del procesador:** Registros de CPU, PC (contador de programa) y flags de estado.
3. **Información de control del proceso:**
    - *Planificación y Estado:* Estado del proceso, prioridad, eventos de espera.
    - *Gestión de Memoria:* Descriptores de segmentos o punteros a tablas de páginas.
    - *Recursos:* Archivos abiertos (descriptores fd), puertos asignados.
    - *Comunicación:* Señales y buzones de mensajería.

2.18. ¿Qué realiza el SO cuando crea un proceso?

Al crear un proceso, el SO realiza los siguientes pasos:

1. **Asignar identificador único:** Inserta una nueva entrada en la tabla primaria de procesos con un PID único.
2. **Reservar espacio en memoria:** Reserva RAM para la imagen del proceso (código, pila y BCP).
3. **Inicializar el BCP:** Establece el PC en la entrada del programa, inicializa registros, punteros de pila y prioridad base.
4. **Establecer enlaces:** Inserta el BCP al final de la cola de planificación correspondiente (ej. cola de Listos).
5. **Estructuras auxiliares:** Genera registros de auditoría y contabilidad de recursos.

2.19. Diferencia fundamental entre un Proceso y un Hilo

- **Proceso:** Unidad de asignación de recursos. Posee su propio espacio de memoria, BCP, archivos abiertos y es independiente de otros procesos.
- **Hilo (Thread):** Unidad de ejecución planificable. Vive dentro de un proceso, comparte su espacio de direcciones y recursos, pero ejecuta su propio flujo con pila y PC individuales.

2.20. Unidad propietaria de recursos vs. Unidad de ejecución

- **Unidad propietaria de recursos (Proceso):** Entidad a la que el SO le asigna un espacio de memoria, archivos abiertos y periféricos.
- **Unidad de ejecución (Hilo):** Entidad con estado de ejecución y contexto de CPU (PC y pila) que el SO planifica para correr en el procesador.

2.21. Multihilo: Concepto y diferencias entre modelos Multihilo y Monohilo

El **multihilo** o *multithreading* es la capacidad de un SO de permitir que un proceso tenga múltiples hilos de ejecución concurrentes.

Comparativa Monohilo vs Multihilo

Característica	Modelo Monohilo	Modelo Multihilo
Definición	Un solo flujo de ejecución por proceso.	Múltiples flujos de ejecución concurrentes en el mismo proceso.
Recursos	Proceso y ejecución son una única entidad.	Hilos comparten el mismo espacio de direcciones y recursos del proceso.
Cambio de contexto	Muy costoso (salva/carga de PCB, tablas de páginas).	Económico y rápido (sólo salva/carga registros de CPU y pila del hilo).
Comunicación	Aislados, requiere IPC (pipes, memoria compartida).	Directa mediante variables globales compartidas.
Gestión	Alta sobrecarga de creación y destrucción.	Muy ligera y rápida.
Protección	Robusto: un fallo en un proceso no afecta a otros.	Frágil: un error fatal en un hilo tira abajo todo el proceso.

2.22. ¿Un hilo puede estar en estado suspendido?

Sí, un hilo puede estar en estado suspendido. La razón técnica reside en que, aunque los hilos comparten recursos del proceso padre, cada uno tiene su propio contexto de ejecución (como el contador de programa y registros). Si el proceso al que pertenece el hilo es llevado a almacenamiento secundario (intercambio o swap) para liberar memoria principal, todos sus hilos quedan suspendidos. Además, un hilo individual podría ser suspendido por el planificador si está esperando que ocurra un evento o si se ha agotado su cuanto de tiempo.

2.23. Tipos de Hilos: ULT y KLT

1. Hilos a Nivel de Usuario (ULT)

Toda la gestión de hilos (creación, planificación y contexto) es realizada por la aplicación mediante una biblioteca en el espacio de usuario. El núcleo no sabe que existen.

- **Ventajas:** Cambio de hilos muy eficiente y rápido (sin pasar a modo núcleo) y planificación personalizada.
- **Desventajas:** Si un hilo se bloquea en una llamada al sistema, bloquea a todo el proceso. Tampoco aprovechan el paralelismo real en multiprocesadores.

2. Hilos a Nivel de Núcleo (KLT)

El núcleo del SO realiza la creación, planificación y destrucción de hilos directamente.

- **Ventajas:** Planificación paralela real en múltiples procesadores. Si un hilo se bloquea, el SO puede planificar otro hilo del mismo proceso.
- **Desventajas:** La conmutación entre hilos es más lenta porque requiere un cambio de contexto a modo núcleo.

3. Threads

3.1. Tareas Simultáneas

Permiten ejecutar varias operaciones al mismo tiempo, optimizando el rendimiento y aprovechando mejor los recursos del sistema.

Orden de Ejecución

El orden de ejecución de los hilos no es determinista; cambia en cada ejecución del programa debido a las condiciones del sistema y la planificación.

Planificación de Hilos

El **scheduler** (planificador) del SO decide qué hilo se ejecuta, asignando tiempo de CPU según prioridades, Round-Robin, entre otros algoritmos.

Efecto de Eliminar Pausas

Sin pausas, un hilo tiende a ejecutarse por completo antes de que otro tome el control, ya que los cambios de contexto ocurren menos frecuentemente. Esto elimina la intercalación visible.

3.2. Starvation (Inanición)

Definición

> *Ocurre cuando un hilo es postergado indefinidamente en el acceso a un recurso compartido porque otros hilos lo acaparan continuamente.*

Inanición vs. Deadlock

- **Deadlock:** Hilos bloqueados mutuamente esperando recursos del otro. Ninguno avanza.
- **Inanición:** El sistema sigue funcionando y otros hilos avanzan, pero un hilo en particular queda postergado.

Mecanismos de Prevención

- **Colas justas (fair locks):** Bloqueos que respetan el orden de espera (ej. `ReentrantLock(true)`).
- **Ajuste de Prioridad:** Elevar temporalmente la prioridad del hilo relegado.
- **Envejecimiento (Aging):** Incrementar la prioridad de un hilo proporcionalmente al tiempo que lleva esperando.

3.3. Sincronización

Conceptos esenciales para coordinar la ejecución multihilo:

- **Mutex / Lock:** Estructuras de exclusión mutua para proteger variables de accesos concurrentes.
- **Semáforo (Semaphore):**(celeste) Variable de control que limita a N el número de hilos accediendo a un recurso.
- **Monitor / Condición (Condition):** Bloques de código para coordinar lógica compleja (ej. Productor/Consumidor).
- **Operaciones Atómicas:** Operaciones a nivel de hardware que garantizan modificaciones sin condiciones de carrera.
- **Sección Crítica (Critical Section):** Bloque de código que accede a recursos compartidos y que solo debe ser ejecutado por un hilo a la vez.

3.4. Race Condition (Condición de Carrera)

> *Ocurre cuando múltiples hilos acceden y modifican un recurso compartido concurrentemente, de modo que el resultado final depende del orden de planificación.*

Inconsistencia en Operaciones

Operaciones simples como `++` no son atómicas (implican leer, modificar y escribir). Si varios hilos ejecutan esto sin sincronización, ocurrirán pérdidas de datos y valores inconsistentes.

Lock vs. Operaciones Atómicas

- **Lock:** Bloquea una sección crítica completa. Seguro, pero introduce sobrecarga por contención.
- **Operaciones Atómicas:** Hacen uso de instrucciones de hardware rápidas (como CAS) sin provocar deadlocks, pero están limitadas a operaciones simples.

3.5. Productor / Consumidor

Necesidad de Sincronización

Sincronizar el productor y el consumidor es vital para evitar que modifiquen la cola de forma concurrente, previniendo condiciones de carrera.

Riesgos de Acceso Libre

Sin sincronización se pueden perder datos, duplicar tareas o provocar errores críticos (escribir en cola llena o leer de cola vacía).

Cola Thread-Safe

Colecciones que gestionan internamente la exclusión mutua, ofreciendo código limpio, legible y libre de deadlocks o race conditions manuales.

3.6. Livelock

Livelock vs. Deadlock

- **Deadlock:** Hilos en espera pasiva (bloqueados).
- **Livelock:** Hilos activos consumiendo CPU y respondiendo mutuamente, pero sin realizar ningún progreso real.

Resolución

- **Backoff Aleatorio:** Esperar un lapso de tiempo aleatorio antes de reintentar la operación, rompiendo la sincronía infinita.
- **Coordinación:** Uso de un coordinador central que asigne turnos.

3.7. Descarga e Interfaz de Usuario

Bloqueo del Hilo Principal

Si una descarga de red se ejecuta en el hilo principal (ui thread), la interfaz se congela, bloqueando clics o cancelaciones por parte del usuario.

Hilo de Interfaz (UI Thread)

En aplicaciones gráficas, el hilo principal debe reservarse para refrescar la interfaz y procesar eventos rápidos. Las tareas lentas deben correr en hilos secundarios.

Tipo de Carga (I/O-Bound vs. CPU-Bound)

Las descargas son tareas ligadas a Entrada/Salida (**I/O-bound**). El microprocesador pasa la mayor parte del tiempo inactivo esperando por la red o disco, por lo que usar hilos mejora sustancialmente la concurrencia.

3.8. Deadlock (Interbloqueo)

Causa del Deadlock

Ocurre cuando cada hilo retiene un recurso y espera otro recurso en posesión del otro, bloqueando el avance del sistema indefinidamente.

Condiciones Necesarias (Coffman)

1. **Exclusión mutua:** Recursos no compartibles.
2. **Retención y espera:** Un hilo retiene un recurso y solicita otro.
3. **Sin desalojo:** El SO no puede quitarle un recurso a un hilo por la fuerza.
4. **Espera circular:** Cadena cíclica de hilos esperándose mutuamente.

Estrategias de Prevención

- **Orden de Bloqueo Jerárquico:** Adquirir recursos siempre en el mismo orden.
- **Timeouts:** Usar bloqueos temporizados (tryLock con timeout) para evitar esperas eternas.
- **Secciones Críticas Reducidas:** Mantener los bloqueos activos el menor tiempo posible.

4. Gestión de Memoria

4.1. División de Memoria en Sistemas Mono y Multiprogramados

1. Sistema Monoprogramado

La memoria principal se divide básicamente en dos partes: * Una reservada para el Sistema Operativo (o monitor residente). * Otra para el único proceso de usuario en ejecución.

El espacio asignado al proceso es contiguo y estático.

2. Sistema Multiprogramado

El SO debe repartir la memoria disponible para hospedar múltiples procesos activos simultáneamente: * **Particiones Fijas:** La memoria se divide en áreas predeterminadas estáticas. * **Particiones Dinámicas:** Regiones de memoria asignadas en tiempo de ejecución con el tamaño exacto solicitado por el proceso. * **Paginación y Segmentación:** Permiten dividir los procesos en bloques pequeños de tamaño fijo o variable, sin requerir contigüidad física. * **Memoria Virtual:** Carga sólo una parte del proceso (conjunto residente) en RAM y mantiene el resto en disco.

4.2. Responsable de la Subdivisión de Memoria

El encargado de administrar y dividir la memoria es el **sistema operativo**. A esta responsabilidad fundamental se le denomina **gestión de la memoria**.

4.3. Particionamiento Fijo

Es el esquema más simple. Consiste en que el SO ocupa una porción fija de la memoria y el resto se reparte en regiones o particiones con límites estáticos.

Alternativas

- **Particiones de igual tamaño:** Toda la memoria libre se divide en particiones que tienen exactamente la misma capacidad.
  - *Dificultades:* Si un programa es muy grande, no entra (requiere overlays); si es muy chico, desperdicia espacio (fragmentación interna).
- **Particiones de distinto tamaño:** La memoria se divide en particiones con tamaños diferentes.
  - *Mejora:* Los procesos grandes se ubican en particiones grandes y los pequeños en reducidas, minimizando la fragmentación interna.

4.4. Limitaciones de las Particiones Fijas

Las principales limitaciones asociadas a las particiones fijas del mismo tamaño son:

- **Fragmentación Interna:** Desperdicio de espacio sobrante dentro de una partición asignada que no puede usar otro proceso.
- **Restricción del tamaño de los procesos:** Procesos que requieren más memoria que la partición más grande del sistema no pueden ejecutarse.
- **Grado de multiprogramación limitado:** El número máximo de procesos activos está limitado por la cantidad de particiones físicas.

4.5. Algoritmo de Ubicación en Particionamiento Fijo

- **Particiones de igual tamaño:** El algoritmo es inmediato. Busca la primera partición libre y la asigna.
- **Particiones de diferente tamaño:**
  - **Mejor ajuste (Best-fit):** Elige la partición libre más pequeña que sirva para minimizar la fragmentación interna.
  - **Primer ajuste (First-fit):** Elige la primera partición que cumpla con el tamaño requerido.

4.6. Particionamiento Dinámico y Compactación

Consiste en un esquema de gestión donde las particiones de memoria son de longitud y número variable.

- **Funcionamiento:** Se crea una partición de forma dinámica con el tamaño exacto solicitado por el proceso.
- **Ventaja:** Elimina la fragmentación interna.
- **Desventaja (Fragmentación Externa):** A medida que los procesos entran y salen, se van generando pequeños huecos libres esparcidos.
- **Solución (Compactación):** El SO desplaza los procesos en memoria para juntar todos los espacios libres en un solo gran bloque contiguo.

4.7. Algoritmos de Ubicación en Particionamiento Dinámico

Se consideran tradicionalmente 3 algoritmos para seleccionar el bloque libre idóneo:

- **Primer ajuste (First-fit):** Recorre la memoria desde el inicio y asigna el primer bloque libre lo suficientemente grande. Es el más rápido.
- **Mejor ajuste (Best-fit):** Busca en toda la lista para seleccionar el hueco cuyo tamaño se ajuste de forma más exacta al tamaño requerido, dejando el menor residuo.
- **Peor ajuste (Worst-fit):** Selecciona el hueco libre más grande disponible en el sistema, bajo la premisa de que el residuo sobrante seguirá siendo lo suficientemente grande como para albergar otro proceso.

4.8. Dirección Lógica vs. Dirección Física

- **Dirección lógica:** Referencia a una ubicación de memoria que generan las instrucciones de un programa. Es totalmente independiente de la asignación real.
- **Dirección física (o dirección absoluta):** Ubicación real y física dentro de la memoria RAM.
- **Traducción:** El hardware (MMU) traduce en tiempo de ejecución las direcciones lógicas en direcciones físicas.

4.9. Funcionamiento de la Paginación y Fragmentación

Técnica que divide los procesos y la memoria física en bloques de tamaño fijo, evitando que un programa requiera almacenamiento contiguo.

Funcionamiento en 3 pasos

1. **División física:** El programa se corta en **páginas** y la RAM en **marcos** del mismo tamaño (potencia de 2).
2. **Tabla de Páginas:** El SO crea un índice para cada proceso indicando qué página del programa está en qué marco de la RAM.
3. **Traducción (MMU):** La unidad de hardware traduce en tiempo de ejecución las direcciones lógicas a direcciones físicas consultando esta tabla.

Fragmentación en Paginación

- **Fragmentación Interna:** SÍ TIENE. El último marco del proceso rara vez se llena por completo, desperdiciando ese residuo.
- **Fragmentación Externa:** NO TIENE. Como todos los marcos son idénticos, cualquier hueco sirve para alojar cualquier página.

4.10. Funcionamiento de la Segmentación y Fragmentación

Técnica que divide los programas en bloques de tamaño variable reflejando su lógica natural (código, pila, datos).

Funcionamiento en 3 pasos

1. **División lógica:** El programa se compone de **segmentos** de distinto tamaño según su función.
2. **Tabla de Segmentos:** Índice que guarda para cada segmento:
   - **Base:** Dirección física de inicio.
   - **Límite:** Tamaño máximo del segmento (por seguridad).
3. **Traducción y control:** El hardware valida que el desplazamiento no supere el límite y le suma la base para hallar la dirección real.

Fragmentación en Segmentación

- **Fragmentación Interna:** **NO TIENE.** Los segmentos se crean con el tamaño justo solicitado.
- **Fragmentación Externa:** **SÍ TIENE.** Al mover segmentos de tamaño variable, la RAM se llena de huecos disjuntos difíciles de aprovechar.

4.11. Ejercicio: Colocación en Particionamiento Dinámico

Configuración Inicial de los Bloques Libres: * B_1: 20M (rango $20 - 40$) * B_2: 60M (rango $80 - 140$) * B_3: 10M (rango $160 - 170$) * B_4: 40M (rango $230 - 270$) * B_5: 30M (rango $290 - 320$) * B_6: 40M (rango $360 - 400$)

Peticiones Secuenciales: $40M, 20M, 10M$.

a) Primer Ajuste (First-Fit)

- **Petición 40M:** Selecciona el primer bloque adecuado desde el inicio: B_2 (60M). Se ubica en el rango $80 - 120$. *Dirección inicial: 80M*. (Deja un residuo libre de 20M en el rango $120 - 140$).
- **Petición 20M:** Selecciona el primer bloque adecuado: B_1 (20M). *Dirección inicial: 20M*. (Bloque B_1 queda totalmente ocupado).
- **Petición 10M:** Selecciona el primer bloque libre adecuado: el residuo de B_2 ($120 - 140$, 20M). *Dirección inicial: 120M*.

Petición	Bloque Asignado	Dirección Inicial
40M	B_2	**80M**
20M	B_1	**20M**
10M	Residuo B_2	**120M**

b) Mejor Ajuste (Best-Fit)

- **Petición 40M:** Busca el bloque más ajustado. Los exactos son B_4 (40M) o B_6 (40M). Elegimos el primero: B_4. *Dirección inicial: 230M*.
- **Petición 20M:** El bloque libre exacto es B_1 (20M). *Dirección inicial: 20M*.
- **Petición 10M:** El bloque libre exacto es B_3 (10M). *Dirección inicial: 160M*.

Petición	Bloque Asignado	Dirección Inicial
40M	B_4	**230M**
20M	B_1	**20M**
10M	B_3	**160M**

c) Peor Ajuste (Worst-Fit)

- **Petición 40M:** Selecciona el bloque más grande disponible: B_2 (60M). Se ubica en el rango $80 - 120$. *Dirección inicial: 80M*. (Deja residuo de 20M).
- **Petición 20M:** Los bloques más grandes disponibles son B_4 (40M) o B_6 (40M). Elegimos B_4. Se ubica en el rango $230 - 250$. *Dirección inicial: 230M*. (Deja residuo de 20M).
- **Petición 10M:** El bloque más grande disponible ahora es B_6 (40M). *Dirección inicial: 360M*. (Deja residuo de 30M).

Petición	Bloque Asignado	Dirección Inicial
40M	B_2	**80M**
20M	B_4	**230M**
10M	B_6	**360M**

5. Memoria Virtual

5.1. Claves de los esquemas de Paginación y Segmentación Sencilla

- **Paginación Sencilla:**
  - Memoria física dividida en marcos fijos y espacio lógico de procesos en páginas.
  - Las páginas pueden reasignarse dinámicamente y ser físicamente no contiguas.
- **Segmentación Sencilla:**
  - Direcciones compuestas por un número de segmento y un desplazamiento.
  - Facilita el manejo de estructuras crecientes, compartición, protección y recompilaciones independientes.

5.2. El Conjunto Residente y sus beneficios de gestión

> ***Conjunto Residente:*** *La porción de la imagen de memoria del proceso que reside físicamente en la memoria principal (RAM) en un instante dado.*

Beneficios de gestión por conjunto residente

- **Aumento de multiprogramación:** Se alojan más procesos en RAM de forma simultánea.
- **Maximización de CPU:** Aumenta la probabilidad de tener siempre procesos listos, reduciendo tiempos muertos.
- **Programas más grandes que la RAM:** Permite ejecutar aplicaciones que exceden la capacidad de la memoria física.
- **Eficiencia:** Evita cargar partes de código rara vez usadas (como rutinas de error).

5.3. La Región de Swap y el Thrashing (Hiperpaginación)

- **Región de Swap:** Zona del disco secundario usada para respaldar la memoria virtual, expulsando páginas inactivas de la RAM.
- **Thrashing (Hiperpaginación):**(rojo) anomalía de rendimiento donde el SO pasa más tiempo intercambiando páginas entre disco y RAM que ejecutando instrucciones útiles.
  - *Causa:* Se asignan muy pocos marcos al proceso para contener su conjunto de trabajo actual, disparando fallos de página continuos.

5.4. El Principio de Proximidad Referencial

> ***Principio de Proximidad Referencial:*** *Establece que un programa en ejecución utiliza en cada momento una pequeña porción de toda la información disponible.*

Es la característica clave que viabiliza y justifica la existencia de la jerarquía de memoria y la memoria virtual.

5.5. Estructura de una Tabla de Páginas y Bits de Control (P y M)

La tabla traduce direcciones lógicas a físicas. Cada entrada cuenta con bits de control clave:

- **Bit P (Presencia):**
  - `P = 1`: La página está cargada en la RAM.
  - `P = 0`: La página está en disco (provoca fallo de página si se accede).
- **Bit M (Modificado):**
  - `M = 1`: La página fue modificada. Al expulsarla, debe escribirse en disco.
  - `M = 0`: La página no cambió y puede descartarse sin escribir.

5.6. Almacenamiento y Optimización de la Tabla de Páginas (TLB)

- **Ubicación:** Las tablas de páginas residen en la RAM.
- **El problema:** Cada acceso a datos virtuales requeriría dos accesos físicos (uno a la tabla y otro al dato).
- **La solución (TLB):** Caché asociativa integrada en la MMU que almacena las traducciones más recientes, reduciendo drásticamente el tiempo de acceso.

5.7. Paginación Multinivel y sus Ventajas

Estructura jerárquica de tablas de páginas útil para grandes espacios de direcciones.

Ventajas

- **Ahorro de RAM:** No requiere cargar las tablas de páginas correspondientes a las zonas vacías de memoria del proceso.
- **Flexibilidad:** Permite que las subtablas crezcan de forma independiente.
- **Residencia parcial:** Sólo la tabla de primer nivel debe residir siempre en la RAM; el resto puede estar en disco.

5.8. Tablas de Páginas Invertidas

- **Concepto:** Una única tabla global con una entrada por cada marco físico de la RAM.
- **Ventaja:** El tamaño de la tabla es proporcional a la RAM y no al número de procesos en ejecución.
- **Desventaja:** Búsqueda lenta (requiere hash) y dificulta la compartición de memoria.

5.9. TLB (Translation Lookaside Buffer)

El **TLB** (Búfer de Traducción Adelantada) es una pequeña memoria caché asociativa de muy alta velocidad que se encuentra integrada directamente en el hardware de la MMU (Unidad de Gestión de Memoria).

5.10. Factores para definir el Tamaño de Página

Hay que considerar principalmente:

- Tamaño de la memoria física.
- Espacio de direcciones lógico.
- Fragmentación interna (cuanto más grande la página, más desperdicio).
- Overhead de las tablas de páginas (páginas pequeñas → tablas más grandes).
- Rendimiento del sistema (accesos a memoria y uso de caché/TLB).

5.11. Implicaciones del Esquema de Segmentación

- **Protección:** Permite permisos lógicos específicos por segmento (ej. código solo lectura).
- **Compartición:** Facilita compartir segmentos lógicos entre procesos.
- **Crecimiento dinámico:** Segmentos como la pila o heap pueden crecer sin solaparse.
- **Inconveniente:** Sufre de fragmentación externa y requiere hardware de traducción complejo.

5.12. Implementación de la Segmentación en Memoria Virtual

En el esquema de segmentación, la implementación se realiza de modo que las referencias a la memoria constan de una dirección virtual compuesta por un número de segmento y un desplazamiento. Para poder llevar el control, se utiliza una tabla de segmentos independiente por cada proceso. Cuando este esquema opera en conjunto con la Memoria Virtual (MV), la tabla utiliza un bit de presencia para cada segmento. Además, la implementación puede requerir otros bits de control si el sistema gestiona la protección o la compartición de los segmentos.

5.13. Segmentación Paginada (Modelo Híbrido)

Combina lo mejor de ambos esquemas:

- **Funcionamiento:** El programa se divide en segmentos lógicos, y cada segmento se divide a su vez en páginas físicas.
- **Traducción:** La tabla de segmentos apunta a tablas de páginas individuales, optimizando la asignación y eliminando la fragmentación externa.

5.14. Paginación bajo Demanda vs. Prepaginación

- **Paginación bajo demanda:** Carga páginas a la RAM sólo cuando se genera un fallo de página al intentar acceder a ellas.
- **Prepaginación:** Carga páginas de forma anticipada basándose en la proximidad espacial (ej. cargar páginas contiguas previendo su uso), optimizando operaciones de disco.

5.15. Importancia de la Ubicación en la Paginación

La ubicación física exacta no es importante para que un programa funcione debido a:

- **Flexibilidad:** Cada marco en RAM puede albergar cualquier página virtual.
- **No contigüidad:** Las páginas de un programa pueden estar dispersas por la RAM.
- **Transparencia:** La MMU traduce direcciones lógicas a físicas de manera invisible para el proceso.
- **Sin fragmentación externa:** Como cualquier página cabe en cualquier marco, no hay huecos inservibles.

5.16. Bloqueo de Marcos de Página

- **Definición:** Bloqueo de marcos específicos para evitar que sean expulsados por la política de reemplazo.
- **Importancia:** Vital para proteger partes críticas como el núcleo (kernel) del SO, buffers de E/S y estructuras de control que requieren disponibilidad inmediata.

5.17. Comparación de los Algoritmos de Reemplazo de Páginas

Algoritmo	Idea básica	Ventajas	Desventajas
FIFO	Reemplaza la página más antigua	Simple de implementar	Puede sacar páginas muy usadas (anomalía de Belady)
Óptimo (OPT)	Reemplaza la que se usará más tarde	Minimiza fallos de página	No se puede implementar en la práctica
LRU	Reemplaza la menos usada recientemente	Buen rendimiento, cercano a OPT	Costoso de implementar
Clock	FIFO con segunda oportunidad (bit de uso)	Buen equilibrio costo/rendimiento	Menos preciso que LRU